

Middleware

By

Jitendra Singh Tomar || Jeetu

Topics to discuss

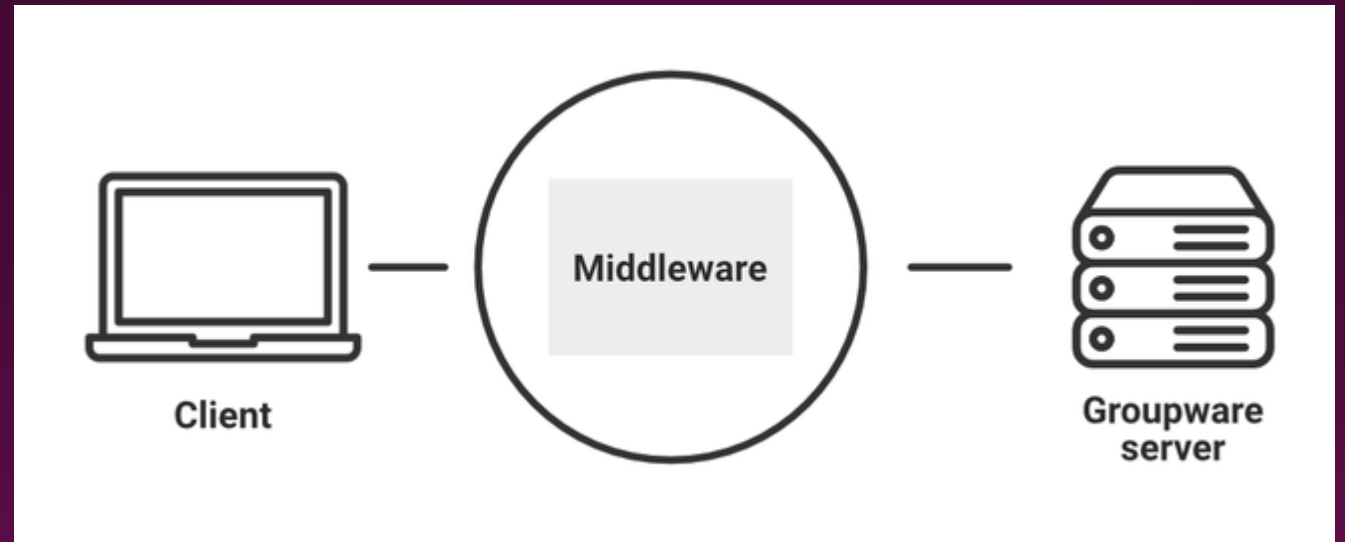
1. Introduction to Webservices
2. Administration of Product Installation and configuration
3. Working with profiles
4. Working with nodes in a Network Deployment/Cluster environment
5. Working with application server
6. Working with clusters

What is Middleware?

- Middleware is software that acts as a bridge between an operating system or database and applications, especially across a distributed network.
- It enables different systems, applications, and components to communicate, coordinate, and manage data and processes.
- In simple terms: **Middleware is the "glue" that connects different applications or systems in a scalable, secure, and reliable way.**

Where Does Middleware Fit?

- It provides essential services like:
 - Communication
 - Messaging
 - Authentication
 - Load balancing
 - Transaction management



Core Functions of Middleware

- **Communication**

- Manages data exchange between applications using protocols (e.g., HTTP, JMS, TCP).

- **Integration**

- Connects heterogeneous systems (e.g., Java app to .NET service).

- **Security**

- Adds authentication, authorization, and encryption.

Core Functions of Middleware

- **Scalability & Load Balancing**

- Distributes traffic to handle growing workloads.

- **Transaction Management**

- Manages multi-step operations reliably (especially in banking, ERP).

- **Logging & Monitoring**

- Tracks performance, errors, and transaction status.

Types of Middleware

Middleware Type	Purpose
Message-Oriented Middleware (MOM)	Asynchronous messaging between systems (e.g., Apache Kafka, ActiveMQ).
Object Request Brokers (ORB)	Facilitates communication between objects in different languages (e.g., CORBA).
Remote Procedure Call (RPC)	Allows calling functions across systems.
Database Middleware	Provides access to and from databases (e.g., ODBC, JDBC).
Web Middleware	Handles web communication, session management (e.g., WebSphere, JBoss).
Enterprise Service Bus (ESB)	Integrates multiple applications via a bus (e.g., MuleSoft, IBM Integration Bus).
API Gateways / Management	Controls API traffic and policies (e.g., Kong, Apigee).
Middleware in Cloud / Containers	Provides middleware-as-a-service (e.g., AWS MQ, Azure Service Bus).

Popular Middleware Platforms

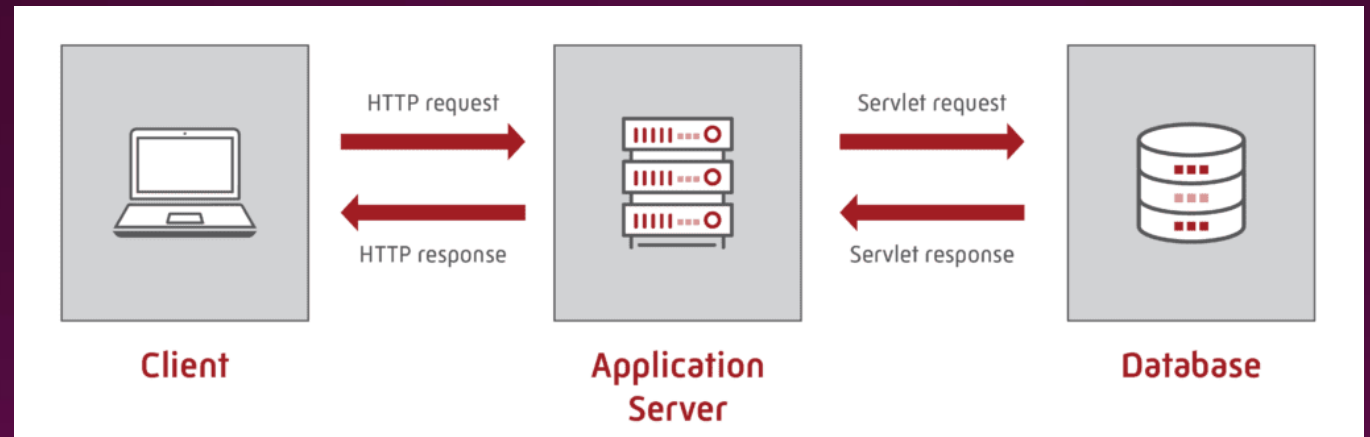
- IBM WebSphere
- Red Hat JBoss EAP
- Oracle WebLogic
- Apache Kafka
- RabbitMQ
- MuleSoft
- TIBCO
- Azure Service Bus
- Amazon MQ / SQS

Why is Middleware Important?

- Reduces **development complexity**.
- Enables **interoperability** across platforms.
- Improves **scalability** and **resilience**.
- Centralizes **security, monitoring, and logging**.
- Supports **distributed systems** and **microservices**.

What is an Application Server?

- An Application Server is a middleware software platform that provides runtime environments, APIs, and services for developing, deploying, and managing enterprise-level applications, especially web-based or distributed applications.



Core Functions:

- **Application Execution**

- Hosts and runs business logic (Java EE, .NET, etc.)

- **Resource Management**

- Manages connections to databases, messaging, etc.

- **Security Services**

- Authentication, authorization, role-based access

- **Session Management**

- Tracks and maintains client session states

- **Transaction Management**

- Ensures atomicity, consistency, isolation, durability (ACID)

- **Web Container**

- Hosts and serves dynamic web content (e.g., JSP, Servlets)

- **Messaging Support**

- Integrates with JMS, MOM for asynchronous communication

Examples of Application Servers

Product	Platform / Tech
IBM WebSphere	Java EE
Oracle WebLogic	Java EE
Red Hat JBoss EAP	Java EE
Apache Tomcat	Lightweight Java servlet container
Microsoft IIS + .NET	Windows/.NET
GlassFish	Open-source Java EE server

What are Integration Endpoints?

- Integration Endpoints are the entry or exit points of a system or service that allow data exchange or interaction between systems, applications, or services in an integration scenario.
- Think of them as connection interfaces exposed by applications or middleware that allow them to communicate with other systems.
- Middleware connects the endpoints, handles how data flows, converts formats (if needed), and ensures everything happens securely and reliably.

Types of Integration Endpoints

Endpoint Type	Description	Example
Web Service Endpoints	SOAP/REST APIs that accept requests and return data	<code>https://api.company.com/orders</code>
Database Endpoints	Integration via direct DB access or stored procedures	JDBC/ODBC connectors
Message Queues / Topics	Publish/subscribe or point-to-point communication	Kafka topic, RabbitMQ queue
FTP/SFTP Endpoints	File-based integration	Uploading invoices to <code>sftp://data.vendor.com/invoices/</code>
Application Adapters	Protocol-specific adapters in ESBs or iPaaS	SAP Adapter, Salesforce Connector
Cloud Service Endpoints	Cloud APIs and webhooks	AWS SNS topic URL, Azure Event Hub

Endpoint Configuration Includes

- **Protocol:** HTTP/S, JMS, TCP, FTP
- **Authentication:** OAuth, JWT, Basic Auth
- **Data Format:** JSON, XML, CSV
- **Endpoint URI/URL:** Target system address
- **Headers and Parameters:** For customization

What Are Profiles in Middleware?

- A profile in middleware is a ready-to-use environment or template that defines
 - how the middleware will operate
 - what services it includes,
 - how it's configured, and
 - what it's used for.
- Profiles Are Used
 - To support different roles or purposes (e.g., development, testing, production)
 - To separate configurations cleanly
 - To enable modular installations
 - To make administration and maintenance easier

Examples of Middleware Profiles

IBM WebSphere Application Server (WAS)

- **Default Profile**

- A typical app server profile with admin console.

- **Admin Profile**

- Only has the administrative tools (no runtime).

- **Managed Profile**

- Used in a cell as a node, controlled by a Deployment Manager.

- **Deployment Manager (DMGR)**

- Controls multiple nodes in a cluster.

- **Custom Profile**

- Lightweight profile created with your specific needs.

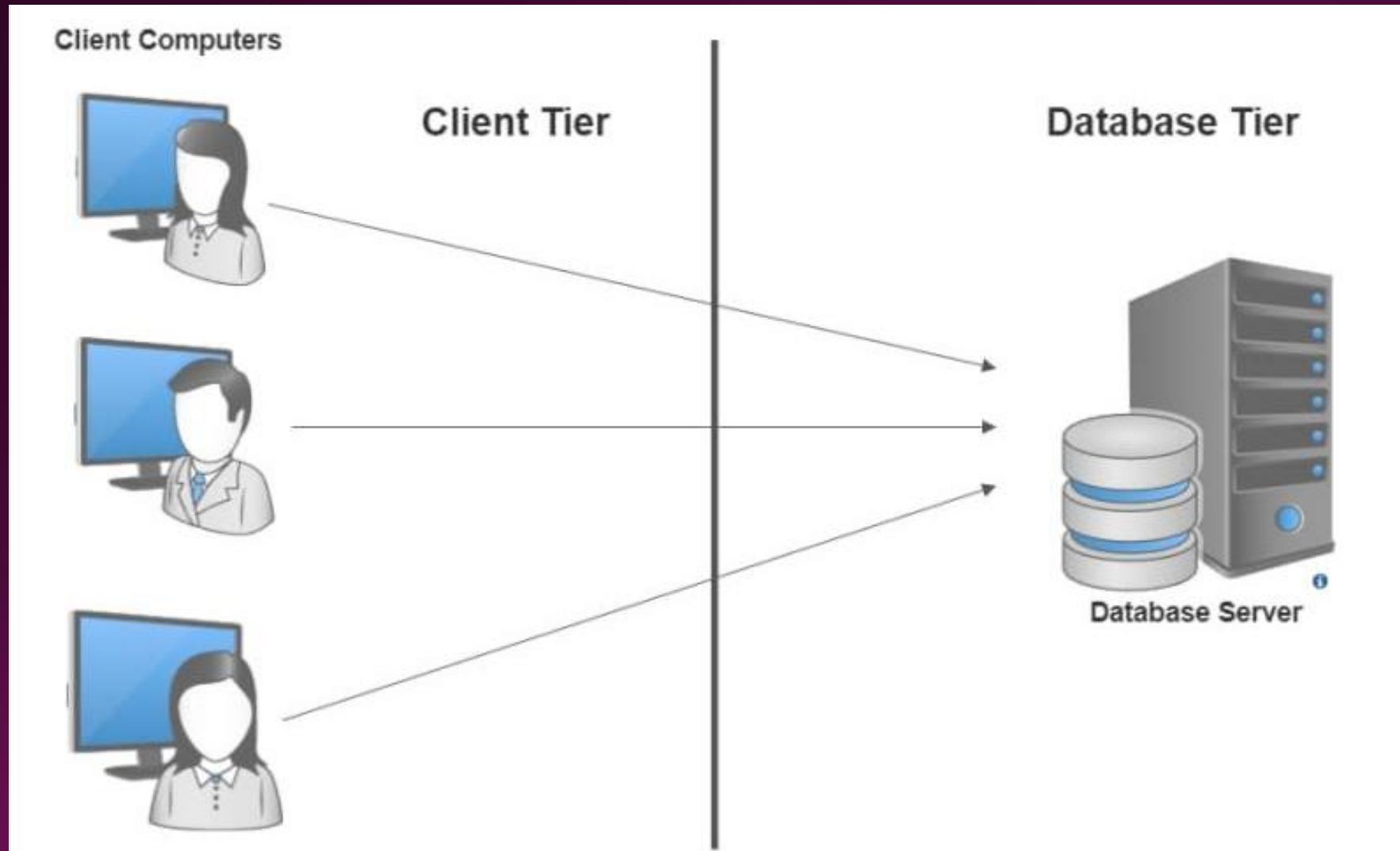
Examples of Middleware Profiles

JBoss / WildFly

- While JBoss doesn't use the word "profiles" in the same sense, it has configuration profiles in its standalone.xml and domain.xml files:

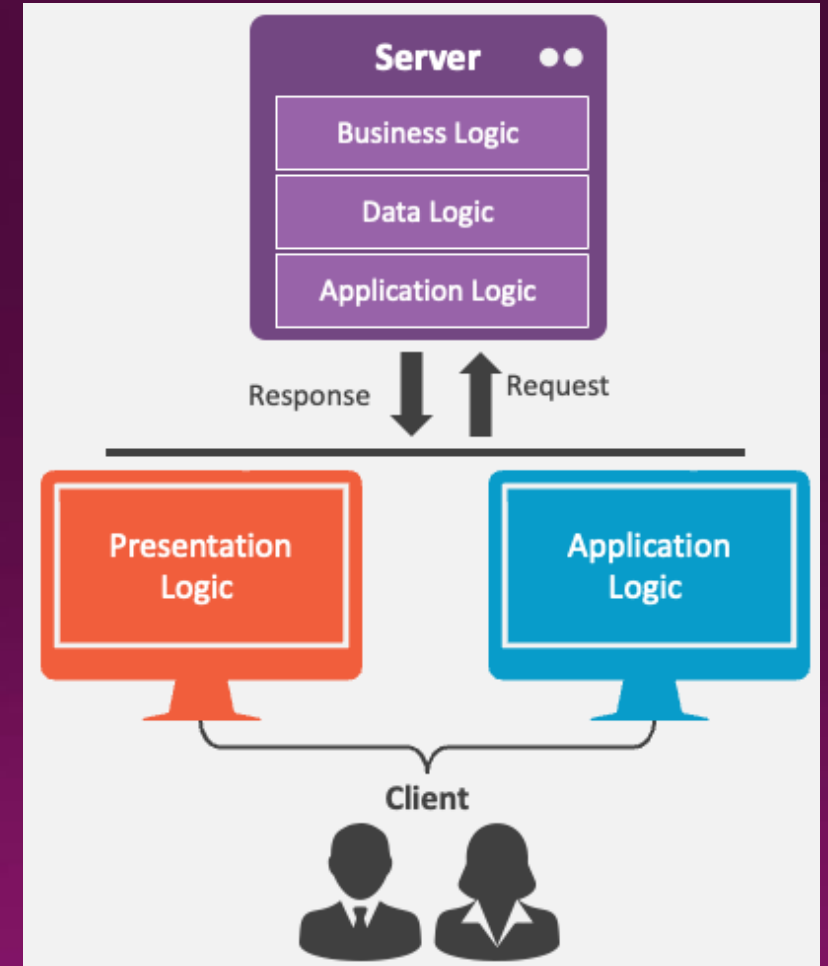
JBoss Profile Type	Description
standalone.xml	Local, single-server profile (good for dev/test).
domain.xml	Centralized config for multiple managed servers.

Two tier architecture



Two tier architecture

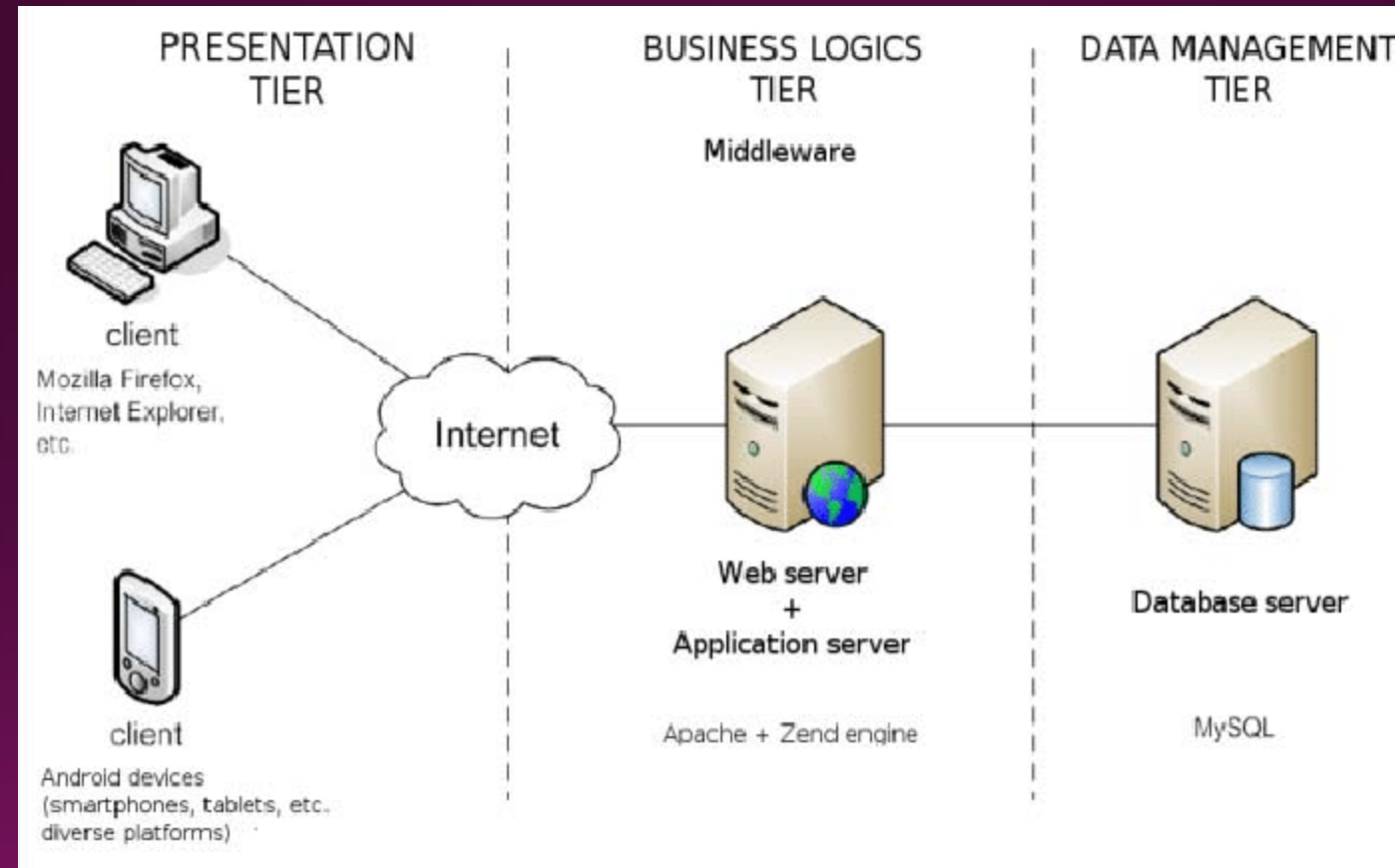
- Two-tier architecture is a client-server model where an application is divided into two layers:
 - Client Tier (Presentation + Logic)
 - Data Tier (Database Server)
- Limitations
 - Harder to scale (clients overload the DB server)
 - Security risks (direct DB access from user machines)
 - Maintenance issues (client-side logic updates are harder to deploy)



Three-Tier Architecture

A three-tier architecture is a software design model that separates an application into three logical layers:

- Presentation Tier
- Application (Logic) Tier
- Data Tier



Three-Tier Architecture – Presentation Tier

Presentation Tier (Client Layer)

- What it does?
 - It's the front-end that users interact with.
- Examples:
 - Web browser, mobile app, desktop app UI.
- Technology:
 - HTML/CSS, JavaScript, Angular, React, Android, iOS.
- Sends requests to the middleware and displays responses to the user.

Three-Tier Architecture – Application Tier

- What it does: This is the middleware layer. It contains the business logic, rules, workflows, and integrations.
- Examples: Application servers, APIs, Web Services, ESBs.
- Technology:
 - Java EE (e.g., JBoss, WebLogic, WebSphere)
 - .NET
 - Node.js
 - Spring Boot
 - MuleSoft, Apache Camel (for integration logic)
- Acts as a bridge between the UI and the database. This is where most of the logic happens.

Three-Tier Architecture – Data Tier

- Data Tier (Database Layer)
- What it does?
 - Stores, retrieves, and manages the application's data.
- Examples:
 - Oracle, MySQL, PostgreSQL, MongoDB.
- Technology:
 - RDBMS, NoSQL, Cloud storage.
- The application tier connects to this layer to fetch, update, or delete data.

